# Major Project - Extended Kalman Filter Implementation for Nonlinear System

(23BME005)

## 1. Objective

To design and implement an Extended Kalman Filter (EKF) for a nonlinear dynamic system to estimate position and velocity from noisy measurements

The objective of this project was to design and implement an Extended Kalman Filter (EKF) for estimating the longitudinal position and velocity of a for a nonlinear dynamic system of a vehicle from noisy sensor data. The core challenge lies in solving the nonlinear equations of motion for a vehicle traversing an inclined road, where it is subjected to driving forces, gravitational components, and aerodynamic drag. Since the sensor suite provides only noisy position measurements, as is typical with GPS systems, the EKF is utilized to "observe" the unmeasured velocity and improve overall estimation accuracy.

The implementation begins by developing a nonlinear dynamics model and generating "ground truth" data through the Runge–Kutta numerical integration method. To simulate a real-world environment, "process" and "measurement" (gaussian) noise are introduced to the system. Because the vehicle's behavior is nonlinear, the EKF linearizes the system at each time step using Jacobian matrices, which approximate the motion around the current state estimate.

By executing a recursive cycle of prediction and update steps, the filter intelligently weights the physics-based model against the noisy sensor data using the Kalman Gain. Ultimately, the performance is evaluated by comparing these filtered estimates against the true vehicle states, demonstrating the EKF's effectiveness in handling partial observability and nonlinearities in a dynamic mechanical system.

## 2. System Model

- <u>State vector:</u>

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Here, x1 represents position and x2 represents velocity.

- <u>Nonlinear dynamics:</u>

$$\dot{x} = \begin{bmatrix} x_2 \\ \dfrac{F - cx_2^2 - mg\sin\theta}{m} \end{bmatrix}$$

This equation describes how position and velocity evolve over time under applied force and drag.

- <u>Measurement model:</u>

$$z = Hx + v \text{, where } H = [1 \quad 0]$$

This means only position is measured, while velocity remains unobserved.

## 3. Noise Modelling

- <u>Process noise:</u>

$$w_k \sim \mathcal{N}(0, Q)$$

This represents uncertainty in the system model.

- <u>Measurement noise:</u>

$$v_k \sim \mathcal{N}(0, R)$$

This represents sensor noise affecting measurements.

## 4. EKF Algorithm

- <u>Prediction Step:</u>

$$\hat{x}_{k|k-1} = \hat{x}_{k-1} + dt\, f(\hat{x}_{k-1})$$

This predicts the next state using system dynamics.

- <u>Jacobian:</u>

$$F_k = \begin{bmatrix} 1 & dt \\ 0 & 1 - \frac{2cx_2}{m} dt \end{bmatrix}$$

This linearizes the nonlinear system around the current estimate.

- <u>Covariance Prediction:</u>

$$P_{k|k-1} = F_k P_{k-1} F_k^T + Q$$

This updates uncertainty in the prediction.

- <u>Update Step:</u>

$$y_k = z_k - H\, x_{k|k-1}$$

This computes the difference between measured and predicted values.

$$S_k = H\, P_{k|k-1}\, H^T + R$$

This represents uncertainty in the measurement prediction.

$$K_k = P_{k|k-1}\, H^T\, S_k^{-1}$$

This determines how much to trust measurements vs prediction.

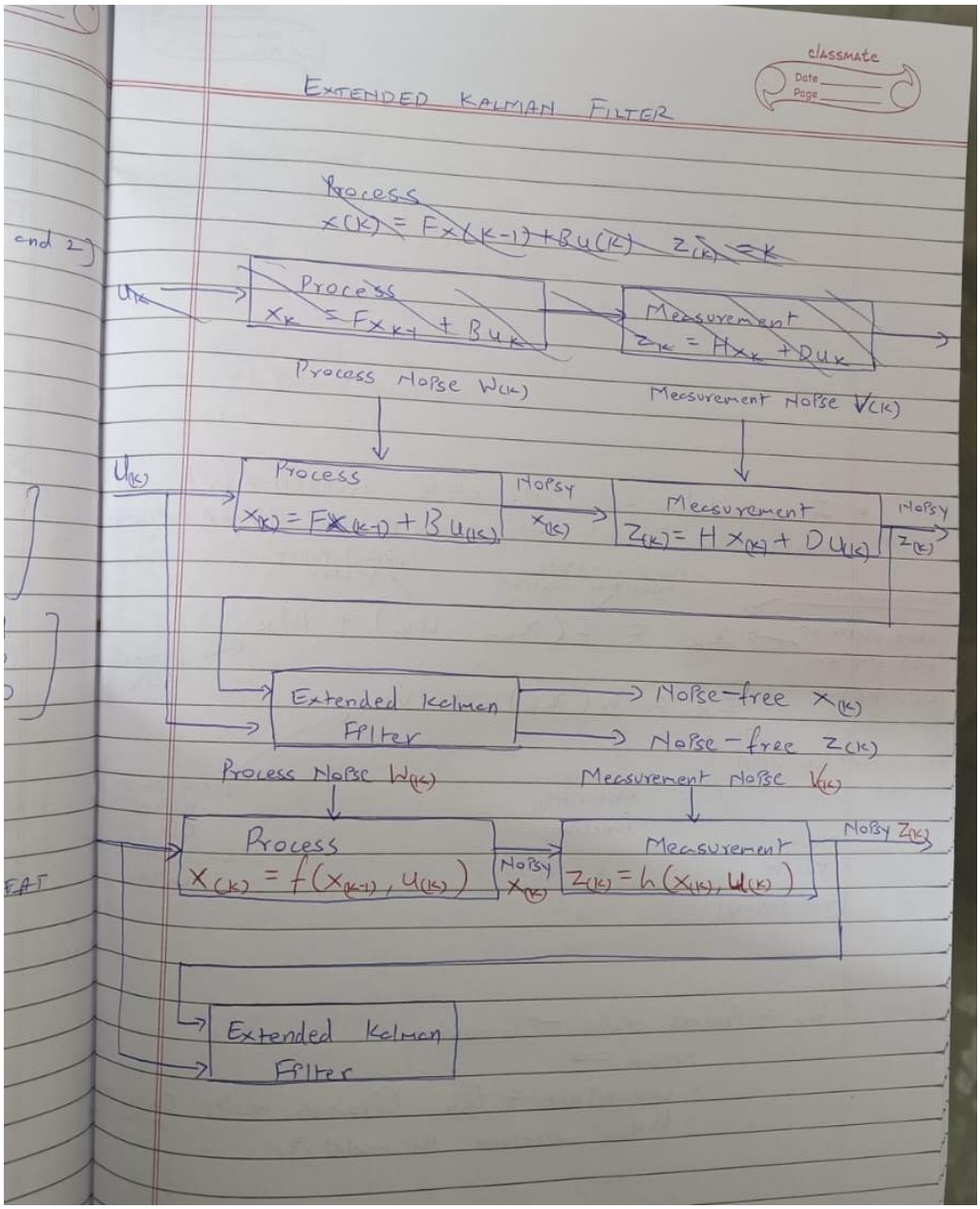
$$\hat{x}_k = \hat{x}_{k|k-1} + K_k\, y_k$$

This updates the state estimate.

$$P_k = (I - K_k H)\, P_{k|k-1}$$

This updates estimation uncertainty.

Handwritten Notes



* In extended KF, the state transition and observation models → need not be linear functions

State (process) model

Nonlinear state transition function

Control input vector

State vector at time step k (ESTIMATE)

$$x_k = f(x_{k-1}, u_k) + w_k$$

process noise

$$z_k = h(x_k) + v_k$$

Measurement (observation) model

nonlinear observation function

Measurement vector (From sensors)

Measurement noise OR Observation noise

$w_k$ = Process Noise → Gaussian

- mean = 0
- covariance = $Q_k$ (Process Noise Covariance Matrix)
  - ↳ How uncertain the model is.

$V_k$ = Measurement noise → Gaussian

- Mean = 0
- Covariance = $R_k$ (Measurement Noise Covariance matrix)

↳ How noisy the sensors are

Q. What are Q and R?

* Noise Covariance Q,

$$Q = E[w w^T]$$

$w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$ $\quad$ w = process noise vector

Q = covariance of that noise

$$\Rightarrow Q = \begin{bmatrix} E[w_1^2] & E[w_1 w_2] \\ E[w_2 w_1] & E[w_2^2] \end{bmatrix}$$

ex: $x = [\text{position} \quad \text{velocity}]^T$ or $\begin{bmatrix} \text{position} \\ \text{velocity} \end{bmatrix}$

~~If system has~~ $x = \begin{bmatrix} p \\ v \end{bmatrix}$

Model: $p_{k+1} = p_k + v_k \Delta t$

$v_{k+1} = v_k$

→ The model assumes zero acc. But if there is some unknown acc, a.

$$\Rightarrow w = \begin{bmatrix} \frac{1}{2} a \Delta t^2 \\ a \Delta t \end{bmatrix} \Rightarrow Q = \begin{bmatrix} E[w_1^2] & E[w_1 w_2] \\ E[w_1 w_2] & E[w_2^2] \end{bmatrix}$$

$$Q = \begin{bmatrix} \sigma_p^2 & \sigma_{pv} \\ \sigma_{vp} & \sigma_v^2 \end{bmatrix}$$

* Measurement Covariance, $R$

$$R = E[v v^T]$$

$v$ = measurement noise vector

$R$ = covariance of that noise

$\therefore R$ = shows relation between sensor noise components.

pf,

$$V = \begin{bmatrix} V_1 \\ V_2 \end{bmatrix} \Rightarrow R = \begin{bmatrix} E[V_1^2] & E[V_1 V_2] \\ E[V_2 V_1] & E[V_2^2] \end{bmatrix}$$

eg Measurement, $z = \begin{bmatrix} x_{GPS} \\ y_{GPS} \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} V_x \\ V_y \end{bmatrix}$

$$\therefore R = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{yx} & \sigma_{y^2} \end{bmatrix}$$

"The sensor is noisy. Possibly its error/noise in $x$ & $y$ → correlated somehow."

# Multidimensional Model 1

Initial state

$x_0$

$P_0$

Previous state

$x_{k-1}$

$P_{k-1}$

New state (Predicted) $k_p$

$$x_{k_p} = A x_{k-1} + B u_k + w_k$$

$$P_{k_p} = A P_{k-1} A^T + Q_k$$

Predicted STATE BASED ON PHYSICAL MODEL AND PREVIOUS STATE

$u$ = CONTROL VARIABLE MATRIX

$w$ = PREDICTED STATE NOISE MATRIX

$Q$ = PROCESS NOISE COVARIANCE MATRIX

$k \to k-1$

Current becomes previous

$I$ = Identity matrix

UPDATE WITH NEW MEASUREMENTS AND KALMAN GAIN

$$K = \frac{P_{k_p} H}{H P_{k_p} H^T + R}$$

$$x_k = x_{k_p} + K\left[Y - H x_{k_p}\right]$$

$$P_k = (I - KH) P_{k_p}$$

$x_p$

OUTPUT OF UPDATED STATE

$x_k$

$P_k$

$x$ = STATE MATRIX ($x^e$, $\dot{x}^p$)

$P$ = PROCESS COVARIANCE MATRIX (Represents errors in the estimated process)

$K$ = Kalman Gain

$R$ = Sensor Noise Matrix (Measurement ERROR)

Covariance Measurement Noise (UNCERTAINTY)

$\Delta T$ = Time for 1 cycle

$$Y_k = C x_{k_m} + Z_k$$

MEASUREMENT INPUT

$Y$ = Measurement of state

# Using Taylor Series

→ Taylor series is used to "approximate a non linear function $f(x)$ around a point".

$f(x) \rightarrow$ approximated using Taylor series.

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \cdots$$

$$\cdots + \frac{f^n(a)(x-a)^n}{n!}$$

EKF uses only first order approximation.

$$\therefore f(x) \approx f(a) + f'(a)(x-a)$$

Therefore, around current estimate $\hat{x}$:

→ state transition,

$$f(x) \approx f(\hat{x}) + F(x-\hat{x})$$

→ Measurement,

$$h(x) \approx h(\hat{x}) + H(x-\hat{x})$$

Here H and F = Jacobian Matrix

Multiple functions of multiple variables → $f(x) = \begin{pmatrix} f_1(x_1, x_2) \\ f_2(x_1, x_2) \end{pmatrix}$

$$J_f = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_2} \end{pmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}_{m \times n}$$

# State :

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \text{position} \\ \text{velocity} \end{bmatrix} \quad -①$$

$$F_{net} = ma \Rightarrow F - cv^2 - mg\sin\theta = F_{net}$$

$$\therefore F_{net} = m\dot{x}_2 = F - cx_2^2 - mg\sin\theta$$

$$\therefore \dot{x}_2 = \frac{F - cx_2^2 - mg\sin\theta}{m} \quad -②$$

$$\dot{x}_1 = x_2 \quad -③$$

Therefore,

$$\dot{x} = f(x) = \begin{bmatrix} x_2 \\ \dfrac{F - cx_2^2 - mg\sin\theta}{m} \end{bmatrix} \quad -④$$

# Discretization

$$x_{k+1} = x_k + \frac{dt}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right) \quad -⑤$$

$$\rightarrow k_1 = f(x_k)$$

$$\rightarrow k_2 = f\left(x_k + \frac{dt}{2} k_1\right)$$

$$\rightarrow k_3 = f\left(x_k + \frac{dt}{2} k_2\right)$$

$$\rightarrow k_4 = f\left(x_k + dt\, k_3\right)$$

The original model ④ is continuous-time, but for computing we need discrete.

Therefore, (simulated) "true value":

$$x_{k+1}^{true} = RK4\,(x_k)$$

# Adding Process Noise:

$$x_{k+1} = RK4(x_k) + w_k$$

# Measurement Noise:

$$z_k = x_{1,k} + V_k$$

$$z_k = H x_k + V_k \quad , \quad H = [1, 0]$$

Here, $\cancel{\begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix}}$ $x := \begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix} = \begin{bmatrix} p \\ v \end{bmatrix}$

We measure only position, velocity = unmeasured

→ We use mvnrnd(mean, covariance) for adding noise

$$w_{-k} = mvn([0,0], Q)$$

(2D vector, Gaussian)

$$v_{-k} = sqrt(R) * randn$$

(1D, scalar)

# EKF Prediction Step:

① State prediction →

$$\hat{x}_{k+1|k} = \hat{x}_k + dt\, f(\hat{x}_k)$$

② Jacobian →

$$\frac{\partial f}{\partial x} = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{2c x_2}{m} \end{bmatrix}$$

$$F_k = \begin{bmatrix} 1 & dt \\ 0 & 1 - \frac{2c x_k}{m} dt \end{bmatrix}$$

③ Covariance Prediction →

$$P_{k+1|k} = F_k P_k F_k^T + Q$$

# EKF Update step

④ Innovation → $\bar{y}_k = z_k - H \hat{x}_{k+1|k}$

⑤ Innovation covariance → $S_k = H P_{k+1|k} H^T + R$

⑥ Kalman Gain →

$$K_k = P_{k+1|k} H^T S_k^{-1}$$

⑦ State Update

$$\hat{x}_{k+1} = \hat{x}_{k+1|k} + K_k \bar{y}_k$$

⑧ Covariance Update

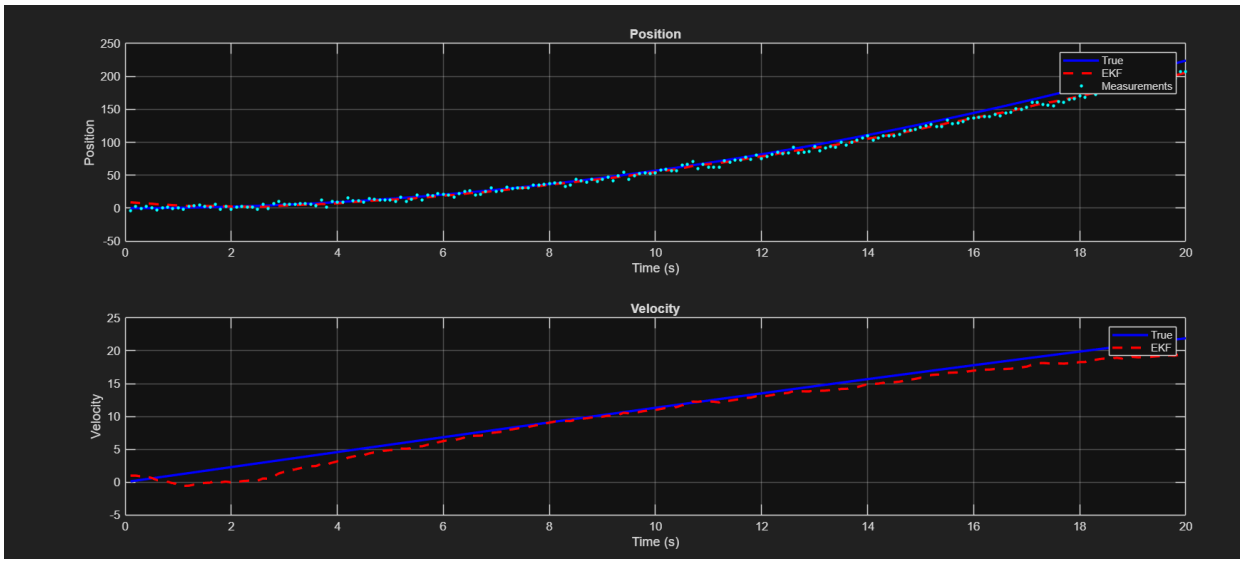
$$P_{k+1} = (I - K_k H) P_{k+1|k}$$

Code and Output

```
clc; clear; close all;

%% parameters
m = 1000;
c = 0.3;
g = 9.81;

theta = 5*pi/180;

dt = 0.1;
T = 20;
N = T/dt;

F = 2000;

%% dynamics: x(1)=pos, x(2)=vel
f_dyn = @(x) [ x(2); (F - c*(x(2)^2) - m*g*sin(theta)) / m];

%% true system (RK4)

x = [0; 0];
X_true = zeros(2, N);

for k = 1:N

    k1 = f_dyn(x);
    k2 = f_dyn(x + 0.5*dt*k1);
    k3 = f_dyn(x + 0.5*dt*k2);
    k4 = f_dyn(x + dt*k3);

    x = x + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);

    X_true(:, k) = x;
end

%% EKF init

x_est = [10; 1];
P = eye(2);

Q = 0.01 * eye(2);
R = 10;

H = [1 0];

X_est = zeros(2, N);
Z = zeros(1, N);

x_sim = [0; 0];

%% main loop

for k = 1:N

    % simulate system (RK4 + noise)
    k1 = f_dyn(x_sim);
    k2 = f_dyn(x_sim + 0.5*dt*k1);
    k3 = f_dyn(x_sim + 0.5*dt*k2);
    k4 = f_dyn(x_sim + dt*k3);

    x_rk4 = x_sim + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);

```

```
    w_k = mvnrnd([0 0], Q)';
    x_sim = x_rk4 + w_k;

    % measurement
    v_k = sqrt(R) * randn;
    z = x_sim(1) + v_k;

    Z(k) = z;

    % prediction (Euler)
    x_dot_est = f_dyn(x_est);
    x_pred = x_est + dt * x_dot_est;

    % Jacobian
    v = x_est(2);

    Fk = [1 dt; 0 1 - (2*c*v/m)*dt];

    % covariance prediction
    P_pred = Fk * P * Fk' + Q;

    % Kalman gain
    S = H * P_pred * H' + R;
    K = P_pred * H' / S;

    % update
    y_tilde = z - H * x_pred;
    x_est = x_pred + K * y_tilde;

    % covariance update
    P = (eye(2) - K * H) * P_pred;

    % store
    X_est(:, k) = x_est;

end

%% plotting

t = (1:N) * dt;

figure;

% position
subplot(2,1,1)

plot(t, X_true(1,:), 'b', 'LineWidth', 2); hold on;
plot(t, X_est(1,:), 'r--', 'LineWidth', 2);
plot(t, Z, 'c.', 'MarkerSize', 8);

legend('True','EKF','Measurements');
title('Position');
xlabel('Time (s)');
ylabel('Position');
grid on;

% velocity
subplot(2,1,2)

plot(t, X_true(2,:), 'b', 'LineWidth', 2); hold on;
plot(t, X_est(2,:), 'r--', 'LineWidth', 2);

legend('True','EKF');
title('Velocity');
xlabel('Time (s)');
```

```
ylabel('Velocity');
grid on;

```



## State Vector x (2x1)

```
x = [ x1
      x2 ]
```

x1 = position
x2 = velocity

## System Function f(x) (2x1)

```
f(x) = [                    x2
        (F - c*x2^2 - m*g*sin(theta))/m ]
```

## Process Noise w_k (2x1)

```
w_k = [ w1
        w2 ]
```

w_k ~ N(0, Q)

## Process Noise Covariance Q (2x2)

```
Q = [ 0.01    0
        0    0.01 ]
```

## Measurement Matrix H (1x2)

```
H = [ 1   0 ]
```

## Measurement Noise v_k (scalar)

v_k ~ N(0, R)

R = [1]

## State Estimate x_hat (2x1)

```
x_hat = [ x1_hat
          x2_hat ]
```

## Covariance Matrix P (2x2)

```
P = [ var(x1)    cov(x1,x2)
      cov(x2,x1) var(x2) ]
```

## Jacobian F_k (2x2)

```
F_k = [ 1              dt
        0     1 - (2*c*x2/m)*dt ]
```

## Kalman Gain K_k (2x1)

```
K_k = [ K1
        K2 ]
```

## Identity Matrix I (2x2)

```
I = [ 1  0
      0  1 ]
```